

Projektaufgabe 2

Phase 2 – Abschluss des Projekts (2.5 P)

Datenmanagement jenseits von Relationen

Gruppen Nummer (A8)

Acikyol Aleyna, 12032843

Özcan Ahmet, 12311287

Sadykova Nargiz, 12129337

May 12, 2026

Dieses Reporting Template dient der Vorbereitung der Abgabe von Phase 2.

Alternativer Import für Ansatz 2 (0.5 Punkte)

Zeigen Sie das create table Statement für A oder B .

```
CREATE TABLE A_ROW (  
    i INT NOT NULL,  
    row DOUBLE PRECISION[] NOT NULL  
);
```

Zeigen Sie, wie die Matrizen A und B des Toy Examples für Ansatz 2 in der DB aussehen.

Reminder:

Matrix A:

$$\begin{bmatrix} 1 & 0 & 3 \\ 0 & 7 & 8 \\ 9 & 8 & 0 \\ 4 & 0 & 2 \end{bmatrix}$$

Matrix B:

$$\begin{bmatrix} 5 & 0 & 7 & 10 & 4 \\ 8 & 9 & 1 & 0 & 8 \\ 4 & 0 & 0 & 2 & 1 \end{bmatrix}$$

Query

Query History

1

SELECT * FROM public.a_row

2

Data Output

Messages

Notifications

Showing rows: 1 to 4

Page No: 1

	i integer	row double precision[]
1	1	{1,0,3}
2	2	{0,7,8}
3	3	{9,8,0}
4	4	{4,0,2}

Figure 1: Matrix A - Reihen

Query

Query History

1

SELECT * FROM public.b_col

2

Data Output

Messages

Notifications

Showing rows: 1 to 5

Page No: 1

	j integer	col double precision[]
1	1	{5,8,4}
2	2	{0,9,0}
3	3	{7,1,0}
4	4	{10,0,2}
5	5	{4,8,1}

Figure 2: Matrix B - Spalten

Ansatz 2 (0.5 Punkte):

Geben Sie den Code der Funktion `dotproduct()` bzw. Ihrer Lösung für Ansatz 2 an.

```
CREATE OR REPLACE FUNCTION dotproduct(a DOUBLE PRECISION[], b DOUBLE PRECISION[])
RETURNS DOUBLE PRECISION AS $$
DECLARE
    result DOUBLE PRECISION := 0.0;
BEGIN
    FOR i IN 1..array_length(a, 1) LOOP
        result := result + a[i] * b[i];
    END LOOP;
    RETURN result;
END;
$$ LANGUAGE plpgsql IMMUTABLE STRICT;
```

Zeigen Sie für das Toy Example, dass C korrekt berechnet wird (z.B. via Screenshot).

```
(1, 1, 17.0)
(1, 2, 0.0)
(1, 3, 7.0)
(1, 4, 16.0)
(1, 5, 7.0)
(2, 1, 88.0)
(2, 2, 63.0)
(2, 3, 7.0)
(2, 4, 16.0)
(2, 5, 64.0)
(3, 1, 109.0)
(3, 2, 72.0)
(3, 3, 71.0)
(3, 4, 90.0)
(3, 5, 100.0)
(4, 1, 28.0)
(4, 2, 0.0)
(4, 3, 28.0)
(4, 4, 44.0)
(4, 5, 18.0)
```

Matrix im
Format [i, j,
val]

```

[[ 17.  0.  7. 16.  7.]
 [ 88. 63.  7. 16. 64.]
 [109. 72. 71. 90. 100.]
 [ 28.  0. 28. 44. 18.]]

```

Umgeformt zur üblichen Matrix mittels
Pandas Dataframe

Erwartete Matrix:

$$C = \begin{pmatrix} 17 & 0 & 7 & 16 & 7 \\ 88 & 63 & 7 & 16 & 64 \\ 109 & 72 & 71 & 90 & 100 \\ 28 & 0 & 28 & 44 & 18 \end{pmatrix}$$

Benchmark Definition (0.5 P):

Als Metrik verwenden wir die **Zeit für eine Matrixmultiplikation** (Durchschnitt über 3 Wiederholungen). Die drei Ansätze werden für alle Kombinationen (L, S) verglichen. Die Matrizen haben die Dimensionen $A \in \mathbb{R}^{(l-1) \times l}$ und $B \in \mathbb{R}^{l \times (l-1)}$.

Parameter	Werte	Kommentar
L	{8, 16, 32, 64, 128, 256}	Zweierpotenzen ab 2^3 ; passt vollständig in den Hauptspeicher
S	{0.1, 0.3, 0.5, 0.7, 0.9}	Sparsity-Werte; Matrix im Allgemeinen eher dicht besetzt

Table 1: Getestete Matrixgrößen L und Sparsity-Werte S (30 Kombinationen)

Auswertung (0.5 P)

Stellen Sie Ihre Messergebnisse grafisch dar.

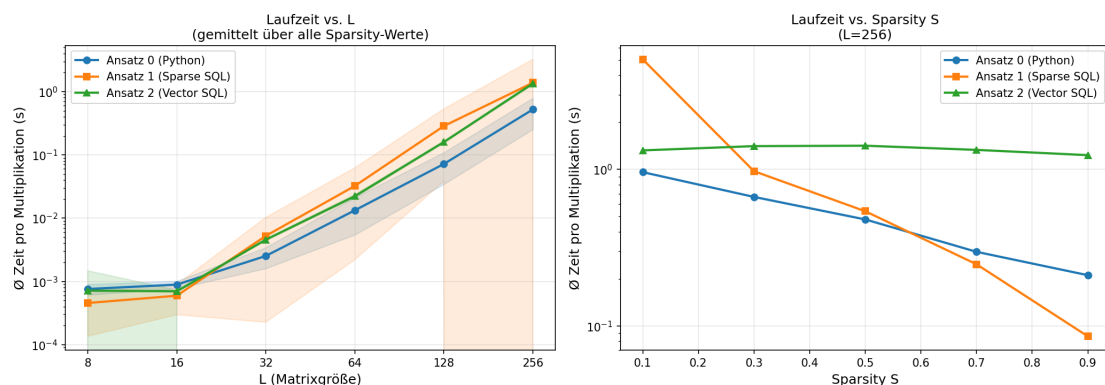


Figure 3: Laufzeit der drei Ansätze: links gemittelt über alle Sparsity-Werte in Abhängigkeit von L ; rechts in Abhängigkeit von S bei $L = 256$.

Fazit: Die Messungen zeigen deutliche Unterschiede im Verhalten der drei Ansätze:

- **Ansatz 1 (Sparse SQL)** profitiert stark von hoher Sparsity: Bei $L = 256$ und $S = 0.1$ (dichte Matrix) benötigt er ca. 5s, bei $S = 0.9$ (dünne Matrix) nur ca. 0.09s. Bei dichter Besetzung ist der Join über die Nicht-Null-Einträge sehr teuer.

- **Ansatz 2 (Vector SQL)** ist weitgehend **sparsity-unabhängig**, da `dotproduct()` stets alle l Array-Elemente durchläuft. Die Laufzeit bei $L = 256$ beträgt konstant ca. 1.3s, unabhängig von S . Bei dichter Matrix ist Ansatz 2 daher besser als Ansatz 1; bei sehr dünner Matrix ist Ansatz 1 deutlich schneller.
- **Ansatz 0 (Python)** überträgt die Daten aus der DB und berechnet das Ergebnis clientseitig. Er überspringt Nulleinträge und skaliert daher ähnlich wie Ansatz 1 mit der Sparsity. Bei $L = 256$ liegt er zwischen beiden DB-Ansätzen und ist bei mittlerer Sparsity konkurrenzfähig.

Insgesamt eignet sich **Ansatz 1** am besten für dünn besetzte Matrizen, während **Ansatz 2** bei dichten Matrizen vorzuziehen ist. **Ansatz 0** zeigt, dass clientseitige Python-Berechnung bei kleinen bis mittleren Matrixgrößen durchaus mit datenbankinternen Ansätzen mithalten kann.

Zeitmanagement

Benötigte Zeit pro Person (nur Phase 2): **Acikyol Aleyna: 6h, Özcan Ahmet: 2h, Sadykova Nargiz: 2 Std.**

References

Important: Reference your information sources!

Remove this section if you use footnotes to reference your information sources.

- Stack Overflow
- <https://www.postgresql.org/docs/current/plpgsql-control-structures.html>
- <https://blog.mclaughlinsoftware.com/2022/04/27/pl-pgsql-array-listing/>
- <https://www.postgresql.org/docs/current/sql-createfunction.html>
- <https://www.postgresql.org/docs/current/plpgsql-development-tips.html>